

Microsoft AZ-204: Developing Solutions for Microsoft Azure - Complete Study Notes

Exam Overview and Official Resources

The **AZ-204** certification validates your ability to design, build, test, and maintain cloud applications and services on Microsoft Azure. Passing earns the **Microsoft Certified: Azure Developer Associate** credential.

Detail	Information
Exam Code	AZ-204
Duration	100 minutes
Passing Score	700 (scale 1-1000)
Format	Multiple choice, case study, drag-and-drop, labs
Skills Measured Date	July 21, 2025

Official Microsoft Links

Resource	URL
Main Exam Page	https://learn.microsoft.com/en-us/credentials/certifications/exams/az-204/
Study Guide (Skills Measured)	https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/az-204
Official Training Course	https://learn.microsoft.com/en-us/training/courses/az-204t00
Practice Assessment	https://learn.microsoft.com/en-us/credentials/certifications/exams/az-204/practice/assessment?assessment-type=practice&assessmentId=35
Exam Sandbox Demo	https://aka.ms/examdemo

Skills Measured with Weightages

Domain	Weight
Develop Azure compute solutions	25–30%
Connect to and consume Azure services	20–25%
Develop for Azure storage	15–20%
Implement Azure security	15–20%
Monitor and troubleshoot Azure solutions	5–10%

Domain 1: Develop Azure Compute Solutions (25-30%)

1.1 Azure App Service Web Apps

Key Concepts

App Service Plan defines compute resources for hosting web apps. Multiple apps can share the same plan.

Pricing Tiers:

- **Free/Shared:** Development only, no SLA
- **Basic (B1-B3):** Dedicated compute, manual scaling up to 3 instances
- **Standard (S1-S3):** Production workloads, autoscaling, deployment slots (up to 5)
- **Premium (P1v2-P3v3):** Enhanced performance, up to 20 slots, 30 instances
- **Isolated:** App Service Environment, maximum isolation, 100 instances

Creating and Configuring Web Apps

```
# Create resource group
az group create --name myResourceGroup --location eastus

# Create App Service plan
az appservice plan create --name myAppServicePlan \
    --resource-group myResourceGroup --location eastus --sku S1

# Create web app
```

```
az webapp create --name MyUniqueAppName \  
  --resource-group myResourceGroup \  
  --plan myAppServicePlan \  
  --runtime "DOTNETCORE:8.0"  
  
# Create with Docker container  
az webapp create -g MyResourceGroup -p MyPlan -n MyAppName \  
  -i myregistry.azurecr.io/docker-image:tag
```

Deployment Methods

ZIP Deploy:

```
az webapp deployment source config-zip \  
  --resource-group myResourceGroup --name myAppName --src ./publish.zip
```

Git Deploy:

```
az webapp deployment source config-local-git \  
  --name MyWebApp --resource-group MyResourceGroup  
  
git remote add azure https://{deploy_user}@MyApp.scm.azurewebsites.net/My  
git push azure master
```

GitHub Actions: Configure in Deployment Center using [azure/webapps-deploy@v2](#)

Configuration Settings

```
# Set configuration  
az webapp config set --name myApp --resource-group myRG \  
  --always-on true --ftps-state FtpsOnly --http20-enabled true \  
  --min-tls-version 1.2 --linux-fx-version "DOTNETCORE|8.0"  
  
# Set app settings (environment variables)  
az webapp config appsettings set \  
  --name myApp --resource-group myRG \  
  --settings SETTING1=value1 SETTING2=value2  
  
# Set connection strings  
az webapp config connection-string set \  
  --name myApp --resource-group myRG \  
  --connection-string "CONNECTIONSTRING=connectionstring"
```

```
--connection-string-type SQLAzure \  
--settings MyDbConnection="Server=..."
```

Deployment Slots

Requirements: Standard tier or higher (Standard: 5 slots, Premium: 20 slots)

Benefits:

- Validate changes before production swap
- Warm-up eliminates cold starts
- Seamless traffic redirection without dropped requests
- Easy rollback by swapping back

```
# Create deployment slot  
az webapp deployment slot create \  
    --name myApp --resource-group myRG --slot staging  
  
# Deploy to slot  
az webapp deployment source config-zip \  
    --name myApp --resource-group myRG --slot staging --src ./app.zip  
  
# Swap slots  
az webapp deployment slot swap \  
    --name myApp --resource-group myRG \  
    --slot staging --target-slot production  
  
# Traffic routing (10% to staging)  
az webapp traffic-routing set \  
    --name myApp --resource-group myRG --distribution staging=10
```

Slot Settings (Sticky): Settings marked "deployment slot setting" remain with the slot during swap.

Scaling

Scale Up (Vertical):

```
az appservice plan update --name myPlan --resource-group myRG --sku P1V2
```

Scale Out (Horizontal):

```
az appservice plan update --name myPlan --resource-group myRG --number-of
```

Autoscale: Configure rules based on CPU percentage, memory, HTTP queue length, or data throughput.

Logging and Diagnostics

```
# Enable application logging
az webapp log config --name myApp --resource-group myRG \
    --application-logging filesystem --level verbose

# Stream logs
az webapp log tail --name myApp --resource-group myRG

# Download logs
az webapp log download --name myApp --resource-group myRG
```

Documentation: <https://learn.microsoft.com/en-us/azure/app-service/overview>

1.2 Azure Functions

Key Concepts

- **Function App:** Container for functions; unit of deployment and management
- **Trigger:** Defines how a function is invoked (each function has exactly ONE trigger)
- **Binding:** Declaratively connects functions to resources (input/output)

Hosting Plans Comparison

Plan	Scaling	Max Instances	Timeout	Key Features
Consumption	Auto, scales to 0	200 (Linux), 100 (Windows)	10 min	Pay per execution, cold starts
Premium (EP1-EP3)	Auto, pre-warmed	100	30 min (configurable)	No cold start, VNet, unlimited duration
Dedicated	Manual/Auto	10-100	Unlimited	Predictable billing, Always On

Triggers and Bindings

Trigger	Description
HTTP	REST API endpoints, webhooks
Timer	Scheduled execution (CRON)
Blob	File uploads to storage
Queue	Messages in Azure Queue Storage
Event Hub	Stream processing
Service Bus	Message queue/topic
Cosmos DB	Change feed processing
Event Grid	Event-driven architecture

Code Examples (C# Isolated Worker)

HTTP Trigger:

```
[Function("HttpExample")]
public static HttpResponseData Run(
    [HttpTrigger(AuthorizationLevel.Function, "get", "post")] HttpRequest
    FunctionContext executionContext)
{
    var response = req.CreateResponse(HttpStatusCode.OK);
    response.WriteString("Hello from Azure Functions!");
    return response;
}
```

Timer Trigger (CRON):

```
[Function("TimerTrigger")]
public void Run([TimerTrigger("0 */5 * * * *")] TimerInfo myTimer)
{
    // Runs every 5 minutes
    _logger.LogInformation($"Timer executed at: {DateTime.Now}");
}
```

Blob Trigger with Output Binding:

```
[Function("BlobTrigger")]
[BlobOutput("output-container/{name}", Connection = "AzureWebJobsStorage")
public byte[] Run(
    [BlobTrigger("input-container/{name}", Connection = "AzureWebJobsStorage")
    string name)
{
    return ProcessBlob(inputBlob);
}
```

Queue Trigger:

```
[Function("QueueTrigger")]
public void Run(
    [QueueTrigger("myqueue-items", Connection = "AzureWebJobsStorage")] string
    FunctionContext context)
{
    var logger = context.GetLogger("QueueFunction");
    logger.LogInformation($"Queue trigger processed: {myQueueItem}");
}
```

Service Bus Trigger:

```
[Function("ServiceBusTrigger")]
public void Run(
    [ServiceBusTrigger("myqueue", Connection = "ServiceBusConnection")] string
    FunctionContext context)
{
    var logger = context.GetLogger("ServiceBusFunction");
    logger.LogInformation($"Message received: {message}");
}
```

Cosmos DB Trigger (Change Feed):

```
[Function("CosmosDBTrigger")]
public void Run(
    [CosmosDBTrigger(
        databaseName: "ToDoList",
        containerName: "Items",
        Connection = "CosmosDBConnection",
        LeaseContainerName = "leases")] IReadOnlyList<MyDocument> documents)
{
}
```

```

foreach (var doc in documents)
{
    _logger.LogInformation($"Document modified: {doc.Id}");
}
}

```

Durable Functions

Function Types:

1. **Client Function:** Entry point, starts orchestrations
2. **Orchestrator Function:** Coordinates workflow (must be deterministic!)
3. **Activity Function:** Unit of work
4. **Entity Function:** Stateful entities

Pattern 1 - Function Chaining:

```

[FunctionName("Chaining")]
public static async Task<object> Run(
    [OrchestrationTrigger] IDurableOrchestrationContext context)
{
    var x = await context.CallActivityAsync<object>("F1", null);
    var y = await context.CallActivityAsync<object>("F2", x);
    var z = await context.CallActivityAsync<object>("F3", y);
    return await context.CallActivityAsync<object>("F4", z);
}

```

Pattern 2 - Fan-out/Fan-in:

```

[FunctionName("FanOutFanIn")]
public static async Task Run(
    [OrchestrationTrigger] IDurableOrchestrationContext context)
{
    var parallelTasks = new List<Task<int>>();
    object[] workBatch = await context.CallActivityAsync<object[]>("GetWorkBatch", null);

    foreach (var item in workBatch)
    {
        Task<int> task = context.CallActivityAsync<int>("ProcessItem", item);
        parallelTasks.Add(task);
    }
}

```

```

    await Task.WhenAll(parallelTasks);
    int sum = parallelTasks.Sum(t => t.Result);
    await context.CallActivityAsync("SendResult", sum);
}

```

Orchestrator Constraints (MUST be deterministic):

- Don't use `DateTime.Now` (use `context.CurrentUtcDateTime`)
- Don't use random numbers
- Don't do I/O directly
- Don't use static variables that change

host.json Configuration

```

{
  "version": "2.0",
  "functionTimeout": "00:10:00",
  "extensions": {
    "queues": {
      "batchSize": 16,
      "maxPollingInterval": "00:00:02"
    },
    "serviceBus": {
      "prefetchCount": 100,
      "maxConcurrentCalls": 32
    },
    "durableTask": {
      "hubName": "MyTaskHub",
      "maxConcurrentActivityFunctions": 10,
      "maxConcurrentOrchestratorFunctions": 5
    }
  }
}

```

CLI Commands

```

# Create Function App (Consumption)
az functionapp create \
  --name myFunctionApp --resource-group myResourceGroup \
  --storage-account mystorageaccount \
  --consumption-plan-location eastus \

```

```
--runtime dotnet --runtime-version 8 --functions-version 4

# Create Function App (Premium)
az functionapp plan create \
    --name myPremiumPlan --resource-group myResourceGroup \
    --location eastus --sku EP1

az functionapp create \
    --name myFunctionApp --resource-group myResourceGroup \
    --storage-account mystorageaccount --plan myPremiumPlan \
    --runtime dotnet

# Azure Functions Core Tools
func init MyFunctionProj --worker-runtime dotnet-isolated
func new --template "HTTP trigger" --name HttpExample
func start --build
func azure functionapp publish myFunctionApp
```

Documentation:

- <https://learn.microsoft.com/en-us/azure/azure-functions/functions-triggers-bindings>
 - <https://learn.microsoft.com/en-us/azure/azure-functions/durable/durable-functions-overview>
-

1.3 Containerized Solutions

Azure Container Registry (ACR)

Tiers:

- **Basic:** Development, cost-optimized
- **Standard:** Production workloads
- **Premium:** Geo-replication, private endpoints, content trust

```
# Create ACR
az acr create --resource-group myResourceGroup \
    --name myContainerRegistry --sku Basic --admin-enabled true

# Login to ACR
az acr login --name myContainerRegistry

# Build image in ACR (no local Docker needed!)
az acr build --registry myContainerRegistry \
```

```
--image sample/myapp:v1 --file Dockerfile .

# List repositories
az acr repository list --name myContainerRegistry --output table

# Import image from Docker Hub
az acr import --name myContainerRegistry \
  --source docker.io/library/nginx:latest --image nginx:latest
```

Authentication Methods: Azure AD, Admin user (not recommended), Service principals (AcrPull, AcrPush roles), Managed identities

Azure Container Instances (ACI)

Features: Serverless containers, fast startup (seconds), per-second billing, Linux and Windows support.

```
# Create container instance
az container create \
  --resource-group myResourceGroup --name mycontainer \
  --image mcr.microsoft.com/azuredocs/aci-helloworld \
  --ports 80 --dns-name-label myapp-demo \
  --location eastus --cpu 1 --memory 1.5

# Create from ACR
az container create \
  --resource-group myResourceGroup --name mycontainer \
  --image myregistry.azurecr.io/myapp:latest \
  --registry-login-server myregistry.azurecr.io \
  --registry-username <service-principal-id> \
  --registry-password <service-principal-password> \
  --ports 80 --dns-name-label myapp

# View logs
az container logs --resource-group myResourceGroup --name mycontainer

# Execute command in container
az container exec --resource-group myResourceGroup \
  --name mycontainer --exec-command /bin/bash
```

Azure Kubernetes Service (AKS)

```
# Create AKS cluster
az aks create \
    --resource-group myResourceGroup --name myAKSCluster \
    --node-count 3 --enable-addons monitoring --generate-ssh-keys

# Attach ACR to AKS
az aks update \
    --name myAKSCluster --resource-group myResourceGroup \
    --attach-acr myContainerRegistry

# Get credentials for kubectl
az aks get-credentials \
    --resource-group myResourceGroup --name myAKSCluster

# kubectl commands
kubectl get nodes
kubectl get pods
kubectl apply -f deployment.yaml
```

Dockerfile Example (Multi-stage .NET)

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY ["MyApp.csproj", "."]
RUN dotnet restore
COPY . .
RUN dotnet publish -c Release -o /app/publish

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish .
EXPOSE 80
ENTRYPOINT ["dotnet", "MyApp.dll"]
```

Documentation: <https://learn.microsoft.com/en-us/azure/container-registry/>

Domain 2: Develop for Azure Storage (15-20%)

2.1 Azure Blob Storage

Storage Account Types and Redundancy

Redundancy	Replicas	Description	Durability
LRS	3 in single datacenter	Lowest cost	11 nines
ZRS	3 across availability zones	High availability	12 nines
GRS	6 (LRS + async secondary)	Disaster recovery	16 nines
RA-GRS	GRS + read from secondary	Read during outage	16 nines
GZRS/RA-GZRS	ZRS primary + LRS secondary	Maximum durability	16 nines

Key Point: ZRS, GZRS, RA-GZRS do **not** support Archive tier.

Blob Types

Type	Max Size	Use Case
Block Blob	~190.7 TiB	Images, videos, documents
Append Blob	195 GB	Log files, streaming
Page Blob	8 TiB	VHDs, VM disks

Access Tiers

Tier	Storage Cost	Access Cost	Min Retention
Hot	Highest	Lowest	None
Cool	Lower	Higher	30 days
Cold	Lower	Higher	90 days
Archive	Lowest	Highest (rehydration)	180 days

Archive tier is offline - must rehydrate (up to 15 hours) before reading.

Lifecycle Management Policies

```
{
  "rules": [{
    "name": "rule1",
```

```

    "enabled": true,
    "type": "Lifecycle",
    "definition": {
      "filters": {
        "blobTypes": ["blockBlob"],
        "prefixMatch": ["container1/log"]
      },
      "actions": {
        "baseBlob": {
          "tierToCool": { "daysAfterModificationGreaterThan": 30 },
          "tierToArchive": { "daysAfterModificationGreaterThan": 90 },
          "delete": { "daysAfterModificationGreaterThan": 180 }
        }
      }
    }
  }
}

```

SDK Usage (Azure.Storage.Blobs)

Client Hierarchy: `BlobServiceClient` → `BlobContainerClient` → `BlobClient`

```

// Using DefaultAzureCredential (recommended)
var blobServiceClient = new BlobServiceClient(
    new Uri("https://<storage-account>.blob.core.windows.net"),
    new DefaultAzureCredential());

// Using Connection String
var blobServiceClient = new BlobServiceClient("<connection-string>");

// Container operations
BlobContainerClient containerClient = await blobServiceClient
    .CreateBlobContainerAsync("mycontainer");

// Upload blob
BlobClient blobClient = containerClient.GetBlobClient("myblob.txt");
await blobClient.UploadAsync("local-file.txt", overwrite: true);

// Download blob
await blobClient.DownloadToAsync("downloaded-file.txt");

// Set access tier
await blobClient.SetAccessTierAsync(AccessTier.Cool);

```

```
// List blobs
await foreach (BlobItem blob in containerClient.GetBlobsAsync())
{
    Console.WriteLine(blob.Name);
}
```

SAS Tokens (Shared Access Signatures)

Type	Signed With	Security Level
User Delegation SAS	Microsoft Entra credentials	Most secure (recommended)
Service SAS	Account key	Medium
Account SAS	Account key	Lowest

```
// Creating User Delegation SAS
UserDelegationKey key = await blobServiceClient.GetUserDelegationKeyAsync(
    DateTimeOffset.UtcNow, DateTimeOffset.UtcNow.AddDays(1));

BlobSasBuilder sasBuilder = new()
{
    BlobContainerName = containerName,
    BlobName = blobName,
    Resource = "b",
    StartsOn = DateTimeOffset.UtcNow,
    ExpiresOn = DateTimeOffset.UtcNow.AddHours(1)
};

sasBuilder.SetPermissions(BlobSasPermissions.Read | BlobSasPermissions.Wr

string sasToken = sasBuilder.ToSasQueryParameters(key, accountName).ToStr
```

CLI Commands

```
# Create storage account
az storage account create --name <storage-account> \
    --resource-group <resource-group> --location eastus \
    --sku Standard_LRS --kind StorageV2

# Upload blob
az storage blob upload --account-name <storage-account> \
```

```
--container-name <container> --name <blob-name> \  
--file <local-file> --auth-mode login
```

```
# Set blob tier
```

```
az storage blob set-tier --account-name <storage-account> \  
--container-name <container> --name <blob-name> --tier Cool
```

```
# Generate user delegation SAS
```

```
az storage container generate-sas --account-name <storage-account> \  
--name <container> --permissions acdlrw --expiry 2024-12-31 \  
--auth-mode login --as-user
```

Documentation: <https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-introduction>

2.2 Azure Cosmos DB

Supported APIs

API	Data Model	Use Case
NoSQL (Core SQL)	JSON documents	New applications, flexible schema
MongoDB	BSon documents	MongoDB migrations
Cassandra	Column-family	Cassandra migrations
Gremlin	Graph	Social networks, recommendations
Table	Key-value	Table Storage migrations

Partitioning (CRITICAL EXAM TOPIC)

Hierarchy:

- **Logical Partition:** Items with same partition key (max 20 GB)
- **Physical Partition:** Azure-managed (max 50 GB, 10,000 RU/s)

Good Partition Key Criteria:

- High cardinality (many distinct values)
- Evenly distributes data and request volume
- Frequently used in WHERE clause
- Immutable

Examples:

- E-commerce: `/customerId`
- IoT: `/deviceId`
- Multi-tenant: `/tenantId`

Consistency Levels

Level	Guarantee	RU Cost	Use Case
Strong	Linearizable	2x	Financial transactions
Bounded Staleness	Reads lag by K ops or T time	2x	Reference data
Session	Read-your-writes in session	1x	Default , most apps
Consistent Prefix	No out-of-order reads	1x	Real-time analytics
Eventual	No ordering guarantee	1x	Low-latency reads

Key Points:

- **Session** is default and most common
- **Strong** not available with multiple write regions
- Strong/Bounded Staleness cost **2x RUs** (read from 2 replicas)

SDK Usage (Microsoft.Azure.Cosmos)

```
// Create client
CosmosClient client = new(
    accountEndpoint: "https://<account>.documents.azure.com:443/",
    tokenCredential: new DefaultAzureCredential());

// Or with connection string
CosmosClient client = new("<connection-string>");

// Create database and container
Database database = await client.CreateDatabaseIfNotExistsAsync("mydb");
ContainerProperties containerProperties = new("mycontainer", "/partitionKey");
Container container = await database.CreateContainerIfNotExistsAsync(
    containerProperties, throughput: 400);

// CRUD Operations
public class Product
{
```

```

    [JsonProperty("id")]
    public string Id { get; set; }
    [JsonProperty("categoryId")]
    public string CategoryId { get; set; } // Partition key
    public string Name { get; set; }
    public decimal Price { get; set; }
}

// CREATE
Product product = new() { Id = "1", CategoryId = "electronics", Name = "I
ItemResponse<Product> response = await container.CreateItemAsync(
    product, new PartitionKey(product.CategoryId));
Console.WriteLine($"RU charge: {response.RequestCharge}");

// READ (Point read - most efficient)
ItemResponse<Product> response = await container.ReadItemAsync<Product>(
    id: "1", partitionKey: new PartitionKey("electronics"));

// UPDATE (Replace)
product.Price = 899.99m;
await container.ReplaceItemAsync(product, product.Id, new PartitionKey(pr

// UPSERT (Create or Update)
await container.UpsertItemAsync(product, new PartitionKey(product.Categor

// DELETE
await container.DeleteItemAsync<Product>("1", new PartitionKey("electroni

// QUERY
QueryDefinition query = new QueryDefinition(
    "SELECT * FROM c WHERE c.categoryId = @category AND c.price < @maxPri
    .WithParameter("@category", "electronics")
    .WithParameter("@maxPrice", 1000);

using FeedIterator<Product> feed = container.GetItemQueryIterator<Product>
while (feed.HasMoreResults)
{
    FeedResponse<Product> response = await feed.ReadNextAsync();
    foreach (var item in response) { Console.WriteLine(item.Name); }
}

```

Change Feed

```

Container leaseContainer = database.GetContainer("leases");

ChangeFeedProcessor processor = container
    .GetChangeFeedProcessorBuilder<Product>("myProcessor", HandleChangesF
    .WithInstanceName("consumerHost1")
    .WithLeaseContainer(leaseContainer)
    .Build();

await processor.StartAsync();

static async Task HandleChangesAsync(
    ChangeFeedProcessorContext context,
    IReadOnlyCollection<Product> changes,
    CancellationToken cancellationTok
{
    foreach (Product item in changes)
    {
        Console.WriteLine($"Changed item: {item.Id}");
    }
}

```

CLI Commands

```

# Create Cosmos DB account
az cosmosdb create --name <account-name> --resource-group <resource-group> \
    --default-consistency-level Session \
    --locations regionName=eastus failoverPriority=0

# Create SQL database
az cosmosdb sql database create --account-name <account-name> \
    --resource-group <resource-group> --name <database-name> --throughput

# Create SQL container
az cosmosdb sql container create --account-name <account-name> \
    --resource-group <resource-group> --database-name <database-name> \
    --name <container-name> --partition-key-path "/categoryId" --throughput

```

Documentation:

- <https://learn.microsoft.com/en-us/azure/cosmos-db/partitioning-overview>
 - <https://learn.microsoft.com/en-us/azure/cosmos-db/consistency-levels>
-

Domain 3: Implement Azure Security (15-20%)

3.1 Microsoft Identity Platform

Authentication Flows

Flow	Use Case	Features
Authorization Code + PKCE	Web apps, SPAs, mobile	Most secure for users
Client Credentials	Server-to-server, daemons	No user, app-only
Device Code	CLI tools, IoT, smart TVs	Input-constrained devices
On-Behalf-Of (OBO)	API calling downstream API	Propagates user identity

Client Credentials Flow: Uses `.default` scope: `https://graph.microsoft.com/.default`

Token Types

Token	Purpose	Lifetime
Access Token	Authorize API calls	~1 hour
ID Token	User authentication info	Short-lived
Refresh Token	Obtain new tokens	24h (SPAs), longer otherwise

Permissions

- **Delegated Permissions:** App acts on behalf of user; effective = app $\cap$ user permissions
- **Application Permissions:** App acts as itself; requires admin consent

MSAL Code Examples

```
// Public Client (Desktop/Mobile)
var app = PublicClientApplicationBuilder
    .Create(clientId)
    .WithAuthority(AzureCloudInstance.AzurePublic, tenantId)
    .WithRedirectUri("http://localhost")
```

```
.Build();

// Interactive authentication
var result = await app.AcquireTokenInteractive(scopes).ExecuteAsync();

// Silent then interactive fallback
var accounts = await app.GetAccountsAsync();
try {
    result = await app.AcquireTokenSilent(scopes, accounts.FirstOrDefault
} catch (MsalUiRequiredException) {
    result = await app.AcquireTokenInteractive(scopes).ExecuteAsync();
}

// Confidential Client (Web Apps/APIs)
var app = ConfidentialClientApplicationBuilder
    .Create(clientId)
    .WithClientSecret(clientSecret)
    .WithAuthority(new Uri($"https://login.microsoftonline.com/{tenantId})
    .Build();

var result = await app.AcquireTokenForClient(scopes).ExecuteAsync();
```

Documentation: <https://learn.microsoft.com/en-us/entra/identity-platform/v2-protocols>

3.2 Managed Identities

System-Assigned vs User-Assigned

Feature	System-Assigned	User-Assigned
Creation	Enabled on resource	Standalone Azure resource
Lifecycle	Tied to resource	Independent
Sharing	One resource only	Multiple resources

DefaultAzureCredential

Authentication Order:

1. EnvironmentCredential
2. WorkloadIdentityCredential
3. ManagedIdentityCredential (in Azure)

4. VisualStudioCredential
5. AzureCliCredential
6. InteractiveBrowserCredential

```
using Azure.Identity;

// Works locally and in Azure
var credential = new DefaultAzureCredential();

// For user-assigned managed identity
var credential = new DefaultAzureCredential(
    new DefaultAzureCredentialOptions {
        ManagedIdentityClientId = "<user-assigned-client-id>"
    });

// Specific credential for production
var credential = new ManagedIdentityCredential(); // System-assigned
var credential = new ManagedIdentityCredential("<client-id>"); // User-as
```

CLI Commands

```
# Enable system-assigned identity on Web App
az webapp identity assign --resource-group myRG --name myApp

# Create user-assigned identity
az identity create --resource-group myRG --name myUserIdentity

# Assign role to managed identity
principalId=$(az webapp identity show --resource-group myRG --name myApp
az role assignment create --role "Key Vault Secrets User" \
    --assignee $principalId \
    --scope /subscriptions/{sub}/resourceGroups/{rg}/providers/Microsoft.Key
```

Documentation: <https://learn.microsoft.com/en-us/entra/identity/managed-identities-azure-resources/overview>

3.3 Azure Key Vault

Key Vault Objects

Object	Purpose
Secrets	Connection strings, passwords
Keys	Cryptographic keys for encryption
Certificates	X.509 certificates for TLS

Access Control

- **Azure RBAC (Recommended):** Fine-grained, key-level permissions
- **Access Policies (Legacy):** Vault-level permissions

Built-in RBAC Roles: Key Vault Administrator, Key Vault Secrets Officer, Key Vault Secrets User

SDK Code Examples

```
using Azure.Identity;
using Azure.Security.KeyVault.Secrets;

string kvUri = $"https://{keyVaultName}.vault.azure.net";
var client = new SecretClient(new Uri(kvUri), new DefaultAzureCredential());

// Set a secret
await client.SetSecretAsync("mySecret", "mySecretValue");

// Get a secret
KeyVaultSecret secret = await client.GetSecretAsync("mySecret");
Console.WriteLine($"Secret value: {secret.Value}");

// Delete a secret
DeleteOperation operation = await client.StartDeleteSecretAsync("mySecret");
await operation.WaitForCompletionAsync();

// List secrets
await foreach (SecretProperties secretProps in client.GetPropertiesOfSecrets())
{
    Console.WriteLine(secretProps.Name);
}
```

Key Vault References in App Service

Format:

```
@Microsoft.KeyVault(SecretUri=https://myvault.vault.azure.net/secrets/mysecret/)
@Microsoft.KeyVault(VaultName=myvault;SecretName=mysecret)
```

Requirements:

1. App has managed identity
2. Identity has Get permission on secrets (or Key Vault Secrets User role)

CLI Commands

```
# Create Key Vault with RBAC
az keyvault create --name "myVault" --resource-group "myRG" \
  --location "eastus" --enable-rbac-authorization true

# Set a secret
az keyvault secret set --vault-name "myVault" \
  --name "MySecret" --value "secretValue"

# Get a secret
az keyvault secret show --vault-name "myVault" --name "MySecret" --query

# Assign RBAC role
az role assignment create --role "Key Vault Secrets User" \
  --assignee "<principal-id>" \
  --scope "/subscriptions/{sub}/resourceGroups/{rg}/providers/Microsoft.KeyVault/vaults/{vaultName}"
```

Documentation: <https://learn.microsoft.com/en-us/azure/key-vault/general/developers-guide>

3.4 Microsoft Graph

Base URL: <https://graph.microsoft.com/v1.0>

GraphServiceClient Examples

```
using Azure.Identity;
using Microsoft.Graph;

// Client Credentials (App-Only)
var scopes = new[] { "https://graph.microsoft.com/.default" };
```

```
var clientSecretCredential = new ClientSecretCredential(tenantId, clientId, clientSecret);
var graphClient = new GraphServiceClient(clientSecretCredential, scopes);

// Get all users
var users = await graphClient.Users.GetAsync();

// Delegated (User Context)
var credential = new InteractiveBrowserCredential(
    new InteractiveBrowserCredentialOptions {
        TenantId = tenantId, ClientId = clientId
    });
var graphClient = new GraphServiceClient(credential, new[] { "User.Read",

// Get current user
var me = await graphClient.Me.GetAsync();

// Read emails
var messages = await graphClient.Me.Messages.GetAsync();
```

Documentation: <https://learn.microsoft.com/en-us/graph/overview>

Domain 4: Monitor and Troubleshoot Azure Solutions (5-10%)

4.1 Application Insights

Telemetry Types

Type	Table	Description
Requests	requests	Incoming HTTP requests
Dependencies	dependencies	External calls (SQL, HTTP)
Exceptions	exceptions	Handled/unhandled exceptions
Traces	traces	Log messages
Custom Events	customEvents	User-defined events
Custom Metrics	customMetrics	User-defined metrics

Configuration

```
// Program.cs (ASP.NET Core)
builder.Services.AddApplicationInsightsTelemetry();

// appsettings.json
{
  "ApplicationInsights": {
    "ConnectionString": "InstrumentationKey=xxx;IngestionEndpoint=https://
  }
}
```

Custom Telemetry with TelemetryClient

```
public class MyController : Controller
{
    private readonly TelemetryClient _telemetryClient;

    public MyController(TelemetryClient telemetryClient)
    {
        _telemetryClient = telemetryClient;
    }

    // Track custom event
    _telemetryClient.TrackEvent("UserLoggedIn",
        new Dictionary<string, string> { { "UserId", userId } },
        new Dictionary<string, double> { { "LoginDuration", durationMs } })

    // Track metric (use GetMetric for pre-aggregation - recommended)
    var metric = _telemetryClient.GetMetric("ItemsInQueue");
    metric.TrackValue(queueLength);

    // Track exception
    try { /* code */ }
    catch (Exception ex)
    {
        _telemetryClient.TrackException(ex,
            new Dictionary<string, string> { { "Operation", "ProcessOrder" } }
    }

    // Track dependency
```

```

var startTime = DateTime.UtcNow;
var timer = Stopwatch.StartNew();
try
{
    // External call
    timer.Stop();
    _telemetryClient.TrackDependency("HTTP", "ExternalAPI", "GET /api
        startTime, timer.Elapsed, true);
}
catch
{
    timer.Stop();
    _telemetryClient.TrackDependency("HTTP", "ExternalAPI", "GET /api
        startTime, timer.Elapsed, false);
    throw;
}
}

```

KQL Query Examples

```

// Failed requests in last 24 hours
requests
| where success == false
| where timestamp > ago(24h)
| summarize count() by resultCode, name
| order by count_ desc

```

```

// Exception tracking
exceptions
| where timestamp > ago(1d)
| summarize count() by type, outerMessage
| top 10 by count_

```

```

// Dependency performance
dependencies
| where timestamp > ago(1h)
| summarize avg(duration) by target, name
| order by avg_duration desc

```

Documentation: <https://learn.microsoft.com/en-us/azure/azure-monitor/app/app-insights-overview>

4.2 Azure Cache for Redis

Cache Tiers

Tier	Features	Use Case
Basic	Single node, no SLA	Dev/test
Standard	Two-node replicated	Production
Premium	Clustering, persistence, VNet	Enterprise

Cache-Aside Pattern

```
public async Task<Product> GetProductAsync(string productId)
{
    string cacheKey = $"product:{productId}";
    string cachedProduct = await _cache.StringGetAsync(cacheKey);

    if (!string.IsNullOrEmpty(cachedProduct))
    {
        return JsonSerializer.Deserialize<Product>(cachedProduct);
    }

    var product = await _database.GetProductAsync(productId);

    if (product != null)
    {
        await _cache.SetStringAsync(cacheKey,
            JsonSerializer.Serialize(product),
            TimeSpan.FromMinutes(30));
    }

    return product;
}
```

StackExchange.Redis Connection

```
using StackExchange.Redis;

// Connection string format
```

```

string connectionString = "myredis.redis.cache.windows.net:6380,password=

// Singleton pattern (recommended)
private static Lazy<ConnectionMultiplexer> lazyConnection = new(() =>
    ConnectionMultiplexer.Connect(connectionString));

public static ConnectionMultiplexer Connection => lazyConnection.Value;
public static IDatabase RedisDb => Connection.GetDatabase();

// Basic operations
await db.StringSetAsync("user:1:name", "John Doe");
string name = await db.StringGetAsync("user:1:name");

// With expiration
await db.StringSetAsync("session:abc", sessionData, TimeSpan.FromHours(1)

// Hash operations
await db.HashSetAsync("user:1", new HashEntry[]
{
    new("name", "John"), new("email", "john@example.com")
});

```

CLI Commands:

```

az redis create --name "myredis" --resource-group "myRG" \
    --location "eastus" --sku "Standard" --vm-size "C1"

az redis list-keys --name "myredis" --resource-group "myRG"

```

Documentation: <https://learn.microsoft.com/en-us/azure/azure-cache-for-redis/cache-overview>

4.3 Azure CDN

CLI Commands

```

# Create CDN profile
az cdn profile create --name "myCDNProfile" --resource-group "myRG" \
    --sku "Standard_Microsoft"

# Create endpoint

```

```
az cdn endpoint create --name "myEndpoint" --resource-group "myRG" \  
    --profile-name "myCDNProfile" --origin "myapp.azurewebsites.net"  
  
# Purge content  
az cdn endpoint purge --name "myEndpoint" --resource-group "myRG" \  
    --profile-name "myCDNProfile" --content-paths '/scripts/*' '/images/*'
```

Documentation: <https://learn.microsoft.com/en-us/azure/cdn/cdn-overview>

Domain 5: Connect to and Consume Azure Services (20-25%)

5.1 Azure API Management

Policy Scopes

```
<policies>  
  <inbound>  
    <!-- Before backend -->  
    <base />  
  </inbound>  
  <backend>  
    <!-- When forwarding to backend -->  
    <base />  
  </backend>  
  <outbound>  
    <!-- After backend response -->  
    <base />  
  </outbound>  
  <on-error>  
    <!-- When errors occur -->  
    <base />  
  </on-error>  
</policies>
```

Common Policies

Rate Limiting:

```
<rate-limit calls="100" renewal-period="60" />
<rate-limit-by-key calls="50" renewal-period="10"
  counter-key="@ (context.Request.IpAddress)" />
```

Validate JWT:

```
<validate-jwt header-name="Authorization" failed-validation-httpcode="401"
  require-scheme="Bearer" output-token-variable-name="jwt-token">
  <openid-config url="https://login.microsoftonline.com/{tenant}/.well-
    <audiences>
      <audience>api://your-app-id</audience>
    </audiences>
    <required-claims>
      <claim name="roles" match="any">
        <value>api.read</value>
      </claim>
    </required-claims>
  </validate-jwt>
```

Cache:

```
<inbound>
  <cache-lookup vary-by-developer="false" vary-by-developer-groups="fal
</inbound>
<outbound>
  <cache-store duration="3600" />
</outbound>
```

Set Header:

```
<set-header name="X-Custom-Header" exists-action="override">
  <value>custom-value</value>
</set-header>
```

Documentation: <https://learn.microsoft.com/en-us/azure/api-management/api-management-policies>

5.2 Azure Event Grid

Event Schema

```
{
  "topic": "/subscriptions/{subId}/...",
  "subject": "/containers/mycontainer/blobs/myblob",
  "eventType": "Microsoft.Storage.BlobCreated",
  "eventTime": "2024-01-15T12:00:00Z",
  "id": "unique-event-id",
  "data": { "api": "PutBlob", "contentLength": 1024 },
  "dataVersion": "1.0"
}
```

Publishing Events

```
using Azure.Messaging.EventGrid;

string topicEndpoint = "https://mytopic.eastus-1.eventgrid.azure.net/api/";
EventGridPublisherClient client = new(
    new Uri(topicEndpoint), new AzureKeyCredential(topicKey));

EventGridEvent myEvent = new(
    subject: "myapp/orders/neworder",
    eventType: "Order.Created",
    dataVersion: "1.0",
    data: new { OrderId = "12345", Amount = 99.99 });

await client.SendEventAsync(myEvent);
```

Event Handler (Azure Function)

```
[FunctionName("EventGridHandler")]
public static void Run(
    [EventGridTrigger] EventGridEvent eventGridEvent, ILogger log)
{
    log.LogInformation($"Event Type: {eventGridEvent.EventType}");
    log.LogInformation($"Data: {eventGridEvent.Data}");
}
```

CLI Commands:

```
az eventgrid topic create --name mytopic --resource-group myRG --location myLocation

az eventgrid event-subscription create --name mysubscription \
  --source-resource-id /subscriptions/{subId}/.../topics/mytopic \
  --endpoint https://myfunction.azurewebsites.net/api/handler
```

Documentation: <https://learn.microsoft.com/en-us/azure/event-grid/overview>

5.3 Azure Event Hubs

SDK Usage

```
using Azure.Messaging.EventHubs;
using Azure.Messaging.EventHubs.Producer;
using Azure.Messaging.EventHubs.Processor;

// Sending events
await using var producer = new EventHubProducerClient(connectionString, eventHubName);
using EventDataBatch batch = await producer.CreateBatchAsync();
batch.TryAddMessage(new EventData("Event 1"));
batch.TryAddMessage(new EventData("Event 2"));
await producer.SendAsync(batch);

// Processing events (production pattern)
BlobContainerClient storageClient = new(storageConnectionString, "checkpoint");
EventProcessorClient processor = new(
    storageClient, EventHubConsumerClient.DefaultConsumerGroupName,
    connectionString, eventHubName);

processor.ProcessEventAsync += async (args) =>
{
    Console.WriteLine($"Event: {args.Data.EventBody}");
    await args.UpdateCheckpointAsync();
};

processor.ProcessErrorAsync += async (args) =>
{
    Console.WriteLine($"Error: {args.Exception.Message}");
};
```

```
await processor.StartProcessingAsync();
```

CLI Commands:

```
az eventhubs namespace create --name myehnamespace --resource-group myRG
  --location eastus --sku Standard

az eventhubs eventhub create --name myeventhub --namespace-name myehnames
  --resource-group myRG --partition-count 4 --message-retention 7
```

Documentation: <https://learn.microsoft.com/en-us/azure/event-hubs/event-hubs-about>

5.4 Azure Service Bus

Queues vs Topics

- **Queues:** Point-to-point, one receiver per message (competing consumers)
- **Topics/Subscriptions:** Publish/subscribe, multiple receivers get copies

Receive Modes

Mode	Description
ReceiveAndDelete	Deleted on receive (high throughput)
PeekLock	Locked, must Complete/Abandon (reliable)

SDK Usage

```
using Azure.Messaging.ServiceBus;

await using ServiceBusClient client = new(connectionString);
ServiceBusSender sender = client.CreateSender(queueName);

// Send message
ServiceBusMessage message = new("Hello, Service Bus!");
message.ApplicationProperties["Priority"] = "High";
await sender.SendMessageAsync(message);
```

```
// Receive with PeekLock
ServiceBusReceiver receiver = client.CreateReceiver(queueName, new ServiceBusReceiverOptions
{
    ReceiveMode = ServiceBusReceiveMode.PeekLock
});

ServiceBusReceivedMessage receivedMessage = await receiver.ReceiveMessageAsync();
Console.WriteLine(receivedMessage.Body.ToString());

// Complete - removes from queue
await receiver.CompleteMessageAsync(receivedMessage);

// Abandon - release lock
await receiver.AbandonMessageAsync(receivedMessage);

// Dead-letter - move to DLQ
await receiver.DeadLetterMessageAsync(receivedMessage, "Reason", "Description");

// Process with ServiceBusProcessor
ServiceBusProcessor processor = client.CreateProcessor(queueName, new ServiceBusProcessorOptions
{
    AutoCompleteMessages = false, MaxConcurrentCalls = 2
});

processor.ProcessMessageAsync += async (args) =>
{
    Console.WriteLine(args.Message.Body.ToString());
    await args.CompleteMessageAsync(args.Message);
};

await processor.StartProcessingAsync();
```

CLI Commands:

```
az servicebus namespace create --name mysbnamespace --resource-group myRG --location eastus --sku Standard

az servicebus queue create --name myqueue --namespace-name mysbnamespace --resource-group myRG --enable-dead-lettering-on-message-expiration true

az servicebus topic create --name mytopic --namespace-name mysbnamespace --resource-group myRG
```

5.5 Azure Queue Storage

```
using Azure.Storage.Queues;

QueueClient queueClient = new(connectionString, queueName);
await queueClient.CreateIfNotExistsAsync();

// Send message
await queueClient.SendMessageAsync("Hello, Queue!");

// With visibility delay
await queueClient.SendMessageAsync("Delayed", visibilityTimeout: TimeSpan.FromMinutes(1));

// Receive and process
QueueMessage[] messages = await queueClient.ReceiveMessagesAsync(maxMessageCount: 10);
foreach (QueueMessage message in messages)
{
    Console.WriteLine(message.Body);

    // Poison message handling
    if (message.DequeueCount > 5)
    {
        // Move to poison queue
        QueueClient poisonQueue = new(connectionString, "myqueue-poison");
        await poisonQueue.SendMessageAsync(message.Body);
    }

    await queueClient.DeleteMessageAsync(message.MessageId, message.PopReceipt);
}
```

Messaging Service Comparison

Feature	Queue Storage	Service Bus	Event Hubs	Event Grid
---------	---------------	-------------	------------	------------

Feature	Queue Storage	Service Bus	Event Hubs	Event Grid
Max Message	64 KB	256 KB (Std)	1 MB	1 MB
Ordering	FIFO not guaranteed	FIFO with sessions	Per-partition	No
Dead-lettering	Manual	Automatic	No	Yes
Pattern	Simple work queue	Enterprise messaging	Stream processing	Event routing

Exam Tips and Common Scenarios

Compute Solutions

- Deployment slots require **Standard tier or higher**
- Autoscale available in Standard+; Automatic scaling requires Premium V2/V3
- Functions Consumption plan has **10-minute timeout** (Premium: unlimited)
- Durable Functions orchestrators **must be deterministic**
- **ACR Build** compiles images in cloud without local Docker

Storage

- Archive tier is **offline** - requires rehydration (up to 15 hours)
- User Delegation SAS is **most secure** (uses Entra ID, not account keys)
- Cosmos DB **Session consistency** is default and most common
- Choose partition key with **high cardinality** that's used in queries

Security

- **DefaultAzureCredential** works both locally (CLI/VS) and in Azure (managed identity)
- Key Vault references format: `@Microsoft.KeyVault(VaultName=x;SecretName=y)`
- Client credentials flow uses `.default` scope
- Authorization Code + PKCE for SPAs (not Implicit flow)

Monitoring

- Application Insights uses **connection string** (not just instrumentation key)
- Use `GetMetric()` for pre-aggregated custom metrics (better performance)
- Redis **cache-aside pattern**: check cache → database → cache result

Messaging

- **Service Bus sessions** for ordered message processing
 - **Event Hubs** for high-throughput streaming (millions/second)
 - **Event Grid** for reactive event routing (Azure resource events)
 - **Queue Storage** for simple work queues when using Storage already
 - APIM `validate-jwt` policy for token validation in inbound section
-

Quick CLI Reference

```
# App Service
az webapp create, az webapp config set, az webapp deployment slot create/

# Functions
az functionapp create, func init, func new, func azure functionapp publish

# Containers
az acr create, az acr build, az container create, az aks create

# Storage
az storage account create, az storage blob upload, az cosmosdb create

# Security
az keyvault create, az keyvault secret set, az webapp identity assign

# Monitoring
az redis create, az cdn endpoint create, az monitor app-insights component create

# Messaging
az servicebus queue create, az eventhubs eventhub create, az eventgrid topic create,
az apim create
```

Last Updated: December 2025 | Skills measured as of July 21, 2025

Primary Source: Microsoft Learn (learn.microsoft.com)

